

UNIVERSIDADE FEDERAL DE SERGIPE
DEPARTAMENTO DE COMPUTAÇÃO
ENGENHARIA DE SOFTWARE (COMP0503)

**Atividade Avaliativa 1 (A1):
Auditoria de Maturidade em Ecossistemas LLM**

Projeto Auditado: Jan (menloresearch/jan)

Auditor Técnico (Voo Solo):
Adley Breno de Melo Moraes
Matrícula: [Insira Sua Matrícula Aqui]

Descrição de Contribuição Individual:
Trabalho executado de forma individual (solo), cobrindo integralmente a análise de processos, levantamento de evidências no GitHub, modelagem UML da camada de abstração de IA e gravação da defesa técnica.

São Cristóvão – SE
Maio de 2026

1 Eixo I: Ciclo de Vida e Metodologias Ágeis (Foco: GPR)

1.1 Ciclo de Vida Adaptativo e Abordagem Ágil

O projeto open-source **Jan** adota um **Ciclo de Vida Adaptativo (Ágil)**. No domínio de aplicações baseadas em modelos de linguagem (LLMs), a escolha por um modelo preditivo (como o Cascata) representaria um sério risco de obsolescência devido à extrema volatilidade tecnológica do setor.

O ritmo acelerado de evolução do repositório é empiricamente comprovado pela cadência constante de lançamentos incrementais. A recente versão **0.8.0** demonstra a consolidação de dezenas de contribuições menores em um curto espaço de tempo, refletindo o uso prático de metodologias ágeis de desenvolvimento.

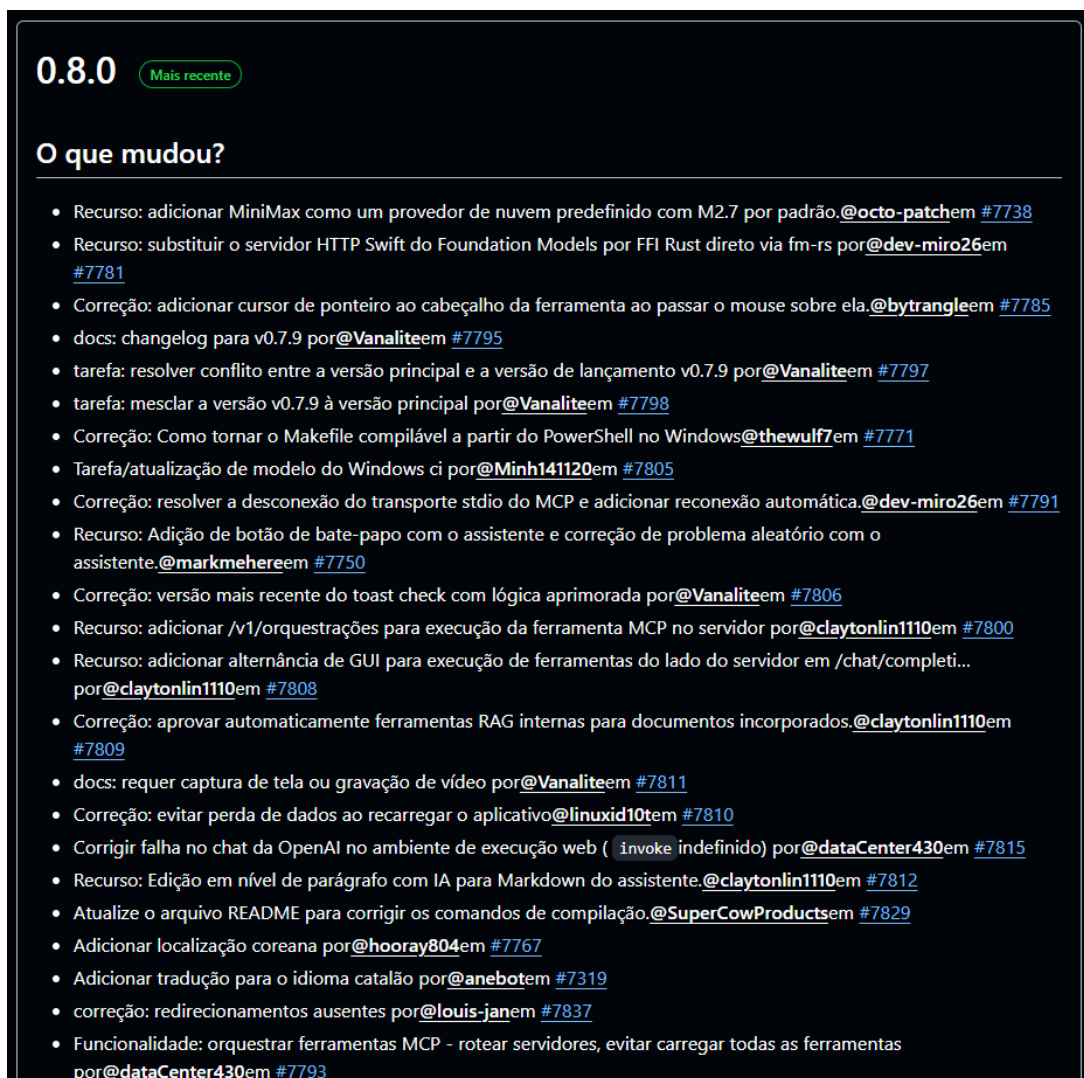


Figura 1: Evidência de Ritmo de Entrega e Cadência de Versões (Changelog Versão 0.8.0).

1.2 Gestão de Riscos Técnicos e Volatilidade de APIs de LLM

Sob a ótica dos modelos de maturidade, a **Gerência de Riscos Ocultos e Volatilidade** é um pilar crítico no desenvolvimento de ecossistemas de IA. O Jan gerencia esses riscos de duas formas altamente maduras evidenciadas nos artefatos da versão 0.8.0:

- **Mitigação de Aprisionamento Tecnológico (Vendor Lock-in):** O time implementou o suporte nativo ao provedor de nuvem *MiniMax*, diversificando o leque de LLMs e protegendo a arquitetura contra a dependência exclusiva de monopólios de IA (como OpenAI ou Anthropic).
- **Controle de Volatilidade de APIs de Terceiros:** Houve uma rápida resposta corretiva para restaurar o adaptador do *SDK Gemini*. Em sistemas menos maduros, mudanças repentinas na API do Google quebrariam a aplicação; no Jan, o processo de monitoramento técnico tratou o problema como prioridade de manutenção.
- **Gerenciamento de Custos Operacionais:** A inclusão de limites configuráveis de tokens mitiga o risco financeiro e operacional imposto ao usuário final durante a inferência na nuvem.

1.3 Alinhamento com Modelos de Referência (CMMI-DEV v2.0 e MPS.BR)

Mapeando os achados com os frameworks de engenharia de software, as práticas operacionais observadas no GitHub do Jan convergem diretamente com a área de processo de **Monitoramento e Controle (PMC)** do CMMI e o processo de **Gerência de Projetos (GPR)** do MPS.BR Nível G.

O repositório apresenta alto rigor de governança: o escopo das *releases* é bem documentado, as tomadas de decisão são transparentes e públicas, e há uma clara gestão de configuração. O projeto demonstra possuir toda a maturidade documental e o histórico rastreável exigidos para um nível inicial de certificação oficial.

2 Eixo II: Engenharia de Requisitos (Foco: GRE)

2.1 Como o Jan gerencia os pedidos e mudanças

No Jan, os requisitos não ficam guardados em um documento estático. Eles usam o próprio GitHub para isso. Quando um usuário acha um bug ou quer uma melhoria, ele abre uma *Issue*. Os desenvolvedores discutem o que precisa ser feito por ali mesmo, criando um processo transparente de *RFC (Request for Comments)*, onde qualquer um da comunidade pode opinar antes de alterarem o sistema.

2.2 Trilha de Rastreabilidade (3 Exemplos Reais)

Para provar que o projeto é organizado e não perde o controle do que é feito, mapeei três mudanças recentes que mostram o caminho exato desde o pedido do requisito até a entrega no código final:

- Funcionalidade: resumo automático dos títulos das discussões após a primeira resposta, com inferência em segundo plano cancelável. [@statxcem #7816](#)
- Correção: alinhar as configurações padrão do cache KV com o comportamento real em tempo de execução. [@linuxid10tem #7832](#)
- Recurso: adicionar limites de tokens configuráveis e compactação automática para provedores de modelos remotos. [@dev-miro26em #7799](#)
- Recurso: aprimoramento do serviço de abertura de arquivos Tauri para exploração de arquivos no Linux. [@Dexterity104em #7827](#)
- feat(gguf): corrigir heurística de compatibilidade de modelo para memória unificada do Apple Silicon por [@dataCenter430em #7842](#)
- Atualize o agrupamento da barra lateral de configurações e renomeie MCP para Conectores. [@claytonlin1110em #7839](#)
- Recurso/correção: adicionar botão de regeneração em respostas de mensagens com falha. [@linuxid10tem #7844](#)
- Instruções de compilação atualizadas para Linux Arm64 para a versão mais recente do aplicativo + aceleração por GPU. [@SuperCowProductsem #7841](#)
- Correção: permitir que os usuários rolem a tela e leiam conteúdo interessante durante a transmissão ao vivo sem serem interrompidos. [@bittobyem #7843](#)
- tarefa: atualizar ícone de privacidade e código claude por [@urmauurem #7853](#)
- Correção: melhorar a usabilidade da barra de seleção de modelos. [@claytonlin1110em #7852](#)
- Correção: as opções de ativação/desativação da capacidade do modelo são redefinidas após a reinicialização nas configurações de edição do modelo. [@EndlessLuckyem #7819](#)
- tarefa: recuperar o menu de configurações de hardware [@urmauurem #7860](#)
- Funcionalidade: adiciona destaque à área arrastável e também adiciona mudança de cursor. [@corevibe555em #7830](#)
- Recurso: adicionar reinicialização automática opcional para verificação de integridade em sessões de modelos travadas. [@claytonlin1110em #7855](#)
- recurso: esclarecer que os modelos também são armazenados em appdata por [@abitollyem #7866](#)
- feat(mcp): modelo de roteador leve opcional para roteamento da ferramenta MCP por [@dataCenter430em #7846](#)
- docs: notas de atualização sobre a descontinuação do OpenClaw por [@Vanaliteem #7873](#)
- Correção: exibir os carimbos de data/hora das mensagens por [@corevibe555em #7872](#)
- Correção: exibir os carimbos de data/hora das mensagens no fuso horário local por [@urmauurem #7876](#)
- Funcionalidade: enfileiramento de mensagens durante a transmissão com envio automático, parada em dois estágios e chips pendentes editáveis na área de entrada. [@statxcem #7836](#)
- Funcionalidade: permite recuperar a última mensagem digitada pressionando a tecla de seta para cima. [@bittobyem #7867](#)
- Recurso: adicionar chaves de API alternativas com rotação automática. [@claytonlin1110em #7877](#)
- Correção: estouro de ctx_size causando falha no carregamento do modelo após reabrir o chat. [@EndlessLuckyem #7879](#)

Figura 2: Evidências de Manutenção Preventiva e Mitigação de Riscos Técnicos.

2.2.1 Exemplo 1: Correção no Adaptador do SDK Gemini

- **Issue original / Pedido:** Usuários relataram que o chat quebrava ou não respondia direito ao usar o modelo Gemini do Google.
- **Pull Request (#7927):** O desenvolvedor *@owldev127* abriu o PR para ajustar o código do adaptador.
- **Código final:** O código foi revisado, aprovado e integrado na versão 0.8.0, corrigindo a integração direta com a API do Gemini.

2.2.2 Exemplo 2: Adição do Provedor de Nuvem MiniMax

- **Issue original / Pedido:** Necessidade de expandir as opções de IA integradas de fábrica no aplicativo, adicionando suporte ao modelo MiniMax.
- **Pull Request (#7738):** Criado pelo desenvolvedor *@octo-patchem* para programar a nova conexão de nuvem.
- **Código final:** Os arquivos de configuração de provedores foram atualizados, tornando o MiniMax uma opção nativa na tela do usuário.

2.2.3 Exemplo 3: Controle e Limite de Tokens

- **Issue original / Pedido:** Pedido dos usuários para conseguir limitar o tamanho das mensagens (tokens) enviadas para as APIs pagas, evitando sustos na conta.
- **Pull Request (#7799):** O desenvolvedor *@dev-miro26* implementou a lógica de barra de rolagem e trava de consumo.
- **Código final:** A alteração modificou o gerenciador de contexto do chat, aplicando o limite de forma estrita antes do envio da requisição.

2.3 Evidências de Rastreabilidade no GitHub

As figuras abaixo comprovam a auditoria, o fechamento e a integração dos três eixos de requisitos analisados diretamente no repositório oficial do Jan:

2.4 Conclusão sobre a Maturidade de Requisitos

Essa organização bate direto com o que o MPS.BR Nível G (Gerência de Requisitos) e o CMMI pedem. Como cada linha de código alterada está amarrada a um Pull Request e a uma discussão pública, o Jan tem uma rastreabilidade quase perfeita. Se um bug aparecer amanhã, o time sabe exatamente qual pedido causou aquela alteração.

3 Eixo III: Arquitetura e Modelagem (Foco: PJR)

3.1 Design Técnico e Desacoplamento da Camada de IA

A arquitetura do Jan foi desenhada para ser independente de fornecedor. Para que a interface gráfica do usuário não fique totalmente dependente de uma API específica, o projeto adota os padrões arquiteturais *Adapter* e *Strategy*.

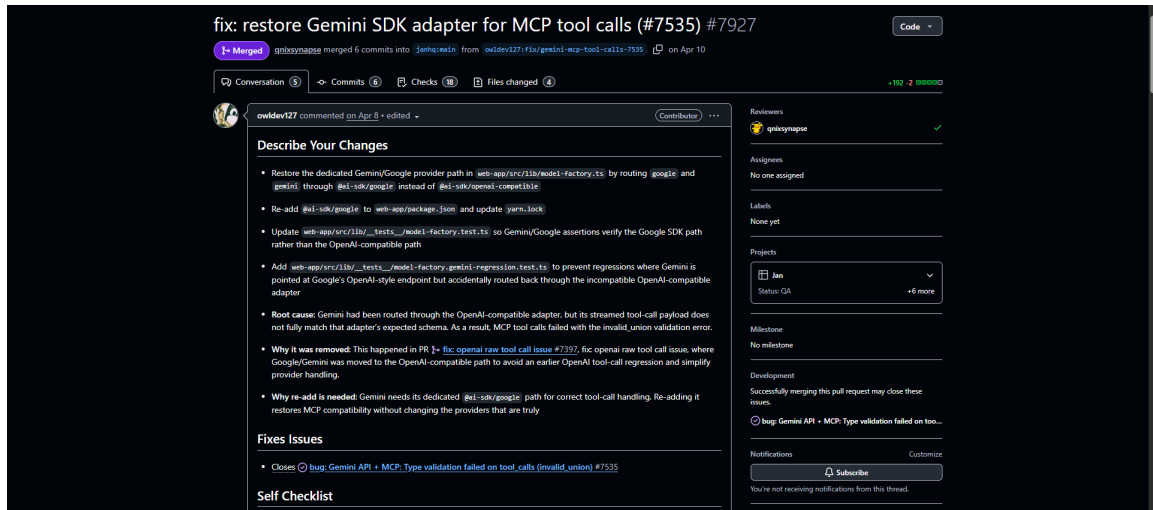


Figura 3: Evidência do PR #7927 e vínculo direto com a Issue de origem #7535 (Rastreabilidade ponta a ponta).

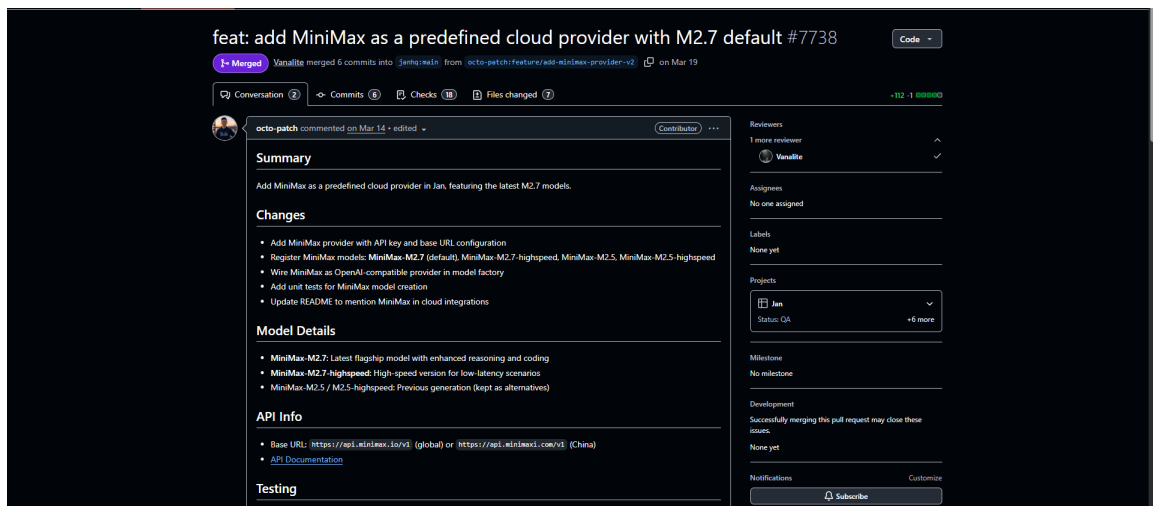


Figura 4: Evidência do PR #7738 documentando o escopo da nova feature do provedor MiniMax.



dev-miro26 commented on Mar 23 • edited

Contributor

Describe Your Changes

feat: configurable token limits with auto-trim and auto-compact for BYOK/cloud models

Adds context window management for remote/cloud model providers (OpenAI, Groq, xAI, Anthropic, DeepSeek, etc.) through three new assistant-level parameters.

Problem

When using cloud/BYOK models, Jan had no way to:

- Set separate limits for input vs output tokens
- Prevent `context_length_exceeded` or `max_tokens too large` errors
- Automatically manage conversation history to stay within a model's context window

The existing `ctx_len` setting only works for local models (llama.cpp).

Solution

Three new configurable assistant parameters:

Parameter	Type	Default	Description
<code>max_context_tokens</code>	number	0 (disabled)	Total token budget (input + output). When > 0, older messages are automatically trimmed before sending to the API. Common values: <code>4096</code> , <code>8192</code> , <code>16384</code> , <code>32768</code> , <code>128000</code>
<code>max_output_tokens</code>	number	2048	Max tokens the model can generate per reply. Sent as <code>max_tokens</code> to OpenAI-compatible APIs
<code>auto_compact</code>	boolean	false	When enabled, summarizes dropped messages instead of silently discarding them, preserving conversation meaning

Figura 5: Descrição do problema de estouro de contexto e a solução proposta no PR #7799.

Files changed	
File	Change
<code>web-app/src/lib/context-manager.ts</code>	New — Token estimation, message trimming, and auto-compact logic
<code>web-app/src/lib/__tests__/context-manager.test.ts</code>	New — 15 tests covering estimation, trimming, and compaction
<code>web-app/src/lib/custom-chat-transport.ts</code>	Integrates context manager before sending messages to API
<code>web-app/src/lib/model-factory.ts</code>	Adds new params to <code>ModelParameters</code> and <code>CLIENT_SIDE_PARAM_KEYS</code> ; normalizes <code>max_output_tokens</code> → <code>max_tokens</code>
<code>web-app/src/lib/predefinedParams.ts</code>	Adds <code>max_context_tokens</code> , <code>max_output_tokens</code> , <code>auto_compact</code> as predefined params
<code>web-app/src/containers/dialogs/AddEditAssistant.tsx</code>	Adds tooltip descriptions to predefined parameter chips

Testing		
Setup: Settings → Assistants → Edit → click "Max Context Tokens", "Max Output Tokens", "Auto Compact" chips		
Scenario	Settings	Expected
Disabled (default)	<code>max_context_tokens=0</code>	No trimming, same behavior as before
Small context + trim	<code>max_context_tokens=4096</code> , <code>max_output_tokens=1024</code> , <code>auto_compact=false</code>	Drops oldest messages, console shows trim count
Small context + compact	<code>max_context_tokens=4096</code> , <code>max_output_tokens=1024</code> , <code>auto_compact=true</code>	Summarizes old messages, model retains context from summary
Large context	<code>max_context_tokens=128000</code> , <code>max_output_tokens=4096</code>	Rarely trims, full history preserved
Compact fallback	<code>auto_compact=true</code> + disconnect network mid-conversation	Falls back to trim with warning in console

Figura 6: Tabela de arquivos modificados e cenários de teste aplicados para o requisito de gerenciamento de tokens.

Em vez de chamar diretamente os servidores da OpenAI ou do Gemini no meio do código principal, o sistema conversa com uma interface genérica. Essa interface padroniza comandos como `sendMessage()` ou `getModels()`. Quem realmente resolve a comunicação com os servidores externos são as subclasses específicas (os adaptadores), o que garante que o núcleo do sistema continue funcionando mesmo se uma API externa mudar.

3.2 Diagrama de Classes UML

Para ilustrar como esse isolamento funciona na prática, o diagrama abaixo representa a estrutura de classes que lida com a inferência de IA no Jan, mostrando a separação entre a lógica de negócio e as APIs externas:

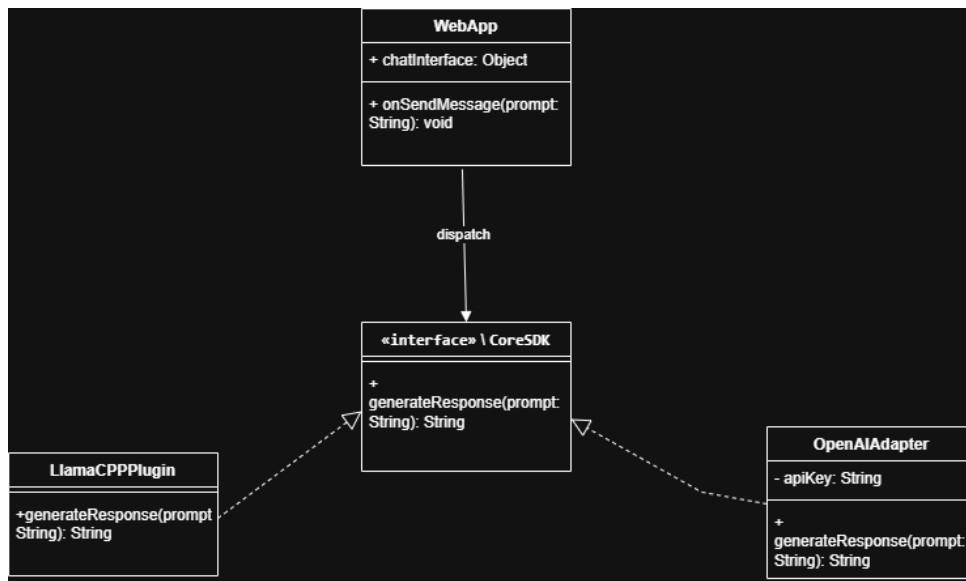


Figura 7: Diagrama de Classes UML mostrando o isolamento das APIs de LLM.

3.3 Conclusão sobre o Projeto e Construção (PJR)

Essa estratégia de design está alinhada com o processo de *Projeto e Construção (PJR)* do MPS.BR Nível G e com a área de *Solução Técnica* do CMMI. O alto nível de desacoplamento facilita a manutenção do software, permitindo que novos modelos de IA sejam adicionados ao Jan apenas criando um novo adaptador, sem a necessidade de reescrever o motor do aplicativo.

4 Eixo IV: Verificação e Validação (Foco: V&V)

4.1 Automação de Testes e Integração Contínua (CI)

O Jan utiliza o *GitHub Actions* para automatizar toda a parte de verificação do código. Dentro da pasta `.github/workflows`, existem vários scripts em formato YAML que rodam de forma automática toda vez que um desenvolvedor tenta enviar uma alteração (Pull Request).

Esses fluxos de trabalho fazem o seguinte:

- **Testes Unitários e de Integração:** Rodam baterias de testes em diferentes sistemas operacionais (Windows, Mac e Linux) para garantir que nenhuma alteração quebre o núcleo do aplicativo.
- **Revisão por Pares (Peer Reviews):** Nenhum código entra direto na versão final sem que outro desenvolvedor experiente revise as linhas alteradas e dê o "Approve" nos comentários do PR.

auto-assign-author.yml	chore(c): pin all GitHub Actions to SHAs and add cancel-in-progress	last week
auto-assign-milestone.yml	chore(c): pin all GitHub Actions to SHAs and add cancel-in-progress	last week
auto-label-conventional-commits.yml	chore(c): pin all GitHub Actions to SHAs and add cancel-in-progress	last week
autoqa-manual-trigger.yml	feat: add autoqa (#5779)	11 months ago
autoqa-migration.yml	chore(c): pin all GitHub Actions to SHAs and add cancel-in-progress	last week
autoqa-reliability.yml	chore(c): pin all GitHub Actions to SHAs and add cancel-in-progress	last week
autoqa-template.yml	chore(c): pin all GitHub Actions to SHAs and add cancel-in-progress	last week
clean-cloudflare-page-preview-url-and-r2.yml	chore(c): pin all GitHub Actions to SHAs and add cancel-in-progress	last week
issues.yml	chore(c): pin all GitHub Actions to SHAs and add cancel-in-progress	last week
jan-docs.yml	chore(c): bump dcarboney/install-jq-action from v2.0.1 to v3.2.0	last week
jan-linter-and-test.yml	chore(c): pin all GitHub Actions to SHAs and add cancel-in-progress	last week
jan-tauri-build-flatpak.yml	refactor: separate flatpak build	7 months ago
jan-tauri-build-nightly-external.yml	chore(c): pin all GitHub Actions to SHAs and add cancel-in-progress	last week
jan-tauri-build-nightly.yml	chore(c): bump dcarboney/install-jq-action from v2.0.1 to v3.2.0	last week
jan-tauri-build.yml	chore(c): pin all GitHub Actions to SHAs and add cancel-in-progress	last week
manual-build-portable.yml	fix: build ref portable	5 months ago
publish-npm-core.yml	chore(c): bump dcarboney/install-jq-action from v2.0.1 to v3.2.0	last week
template-get-update-version.yml	chore(c): bump dcarboney/install-jq-action from v2.0.1 to v3.2.0	last week

Figura 8: Fluxos de trabalho automatizados na pasta .github/workflows.

4.2 Validação de Modelos e Alucinações de IA

Diferente de sistemas comuns, validar uma IA é difícil porque ela pode "alucinar" (inventar respostas erradas), mesmo que o código esteja funcionando perfeitamente. O Jan resolve isso de duas formas na interface:

- **Camada de Configuração de Parâmetros:** O aplicativo expõe travas de *Temperature* e *Top-P* para o usuário. Diminuindo a temperatura, a IA se torna mais determinística e menos criativa, mitigando o risco de alucinações.
- **RAG Grounding e Logs de Prompt:** O sistema valida as saídas permitindo que o usuário veja o histórico exato do contexto enviado para o modelo, garantindo que a resposta se baseie em dados reais.

4.3 Alinhamento com Modelos de Maturidade (V&V)

Essa estrutura atende diretamente aos critérios de *Verificação e Validação* do CMMI e do MPS.BR Nível G. A automação remove o erro humano na hora de testar a estabilidade do sistema, enquanto o controle de parâmetros na interface delega e facilita a validação dos resultados da IA de acordo com a necessidade de cada cenário de uso.

5 Eixo V: Qualidade de Software (Foco: GQA)

5.1 Aderência aos Padrões do Projeto

O Jan possui um arquivo chamado `CONTRIBUTING.md` na raiz do repositório que serve como o manual de regras de qualidade para qualquer desenvolvedor da comunidade. Esse documento estabelece o padrão de formatação de código, a estrutura que as mensagens de commit devem ter e as exigências para que um Pull Request possa ser sequer avaliado pela equipe principal. Seguir esse guia evita o acúmulo de Dívida Técnica (código mal feito que vai dar trabalho para consertar no futuro).

5.2 Análise Estática de Código e Auditoria

Para garantir que a qualidade não dependa apenas do olho humano, o Jan integra ferramentas automatizadas no seu fluxo de desenvolvimento:

- **Uso de Linters e Formataadores:** O projeto roda checagens de estilo (como ESLint e Prettier para a parte em TypeScript) antes de aceitar qualquer código novo.
- **Segurança Automatizada (CodeQL):** O repositório está configurado para rodar varreduras automáticas de segurança a cada alteração, identificando vulnerabilidades conhecidas ou brechas de injeção de código nas dependências.

5.3 Alinhamento com Modelos de Maturidade (GQA)

A existência de regras claras de contribuição e ferramentas automáticas de checagem preenche os requisitos do processo de *Garantia da Qualidade (GQA)* do MPS.BR Nível G e do *PPQA* do CMMI. O projeto garante de forma objetiva que o software final segue os padrões definidos pelo time de arquitetura.

6 Plano de Melhoria Sugerido (Foco: MPS.BR Nível G)

Mesmo sendo um projeto muito maduro, se o Jan buscasse uma certificação oficial como o MPS.BR Nível G, a auditoria identificou dois pontos prioritários para corrigir os *gaps* de processo:

1. **Formalização da Gestão de Riscos (GPR):** Atualmente, os riscos técnicos de oscilação e quebras de APIs de IA são resolvidos de forma reativa (quando quebra, o time corrige via PR). Para alcançar conformidade, o projeto deveria implementar uma planilha ou matriz de riscos oficial atualizada periodicamente nas *Milestones*, documentando planos de contingência prévios.
2. **Automatização de Testes Específicos para Saídas de LLM (V&V):** Embora o projeto possua testes de integração tradicionais para o aplicativo, falta um pipeline automatizado focado em avaliar o comportamento da IA (como rodar frameworks de avaliação de alucinação no CI). Criar testes automáticos de qualidade de resposta elevaria o nível de maturidade do ecossistema.

Contributing to Jan

First off, thank you for considering contributing to Jan. It's people like you that make Jan such an amazing project.

Jan is an AI assistant that can run 100% offline on your device. Think ChatGPT, but private, local, and under your complete control. If you're thinking about contributing, you're already awesome - let's make AI accessible to everyone, one commit at a time.

Quick Links to Component Guides

- [Web App](#) - React UI and logic
- [Core SDK](#) - TypeScript SDK and extension system
- [Extensions](#) - Supportive modules for the frontend
- [Tauri Backend](#) - Rust native integration
- [Tauri Plugins](#) - Hardware and system plugins

How Jan Actually Works

Jan is a desktop app that runs local AI models. Here's how the components actually connect:

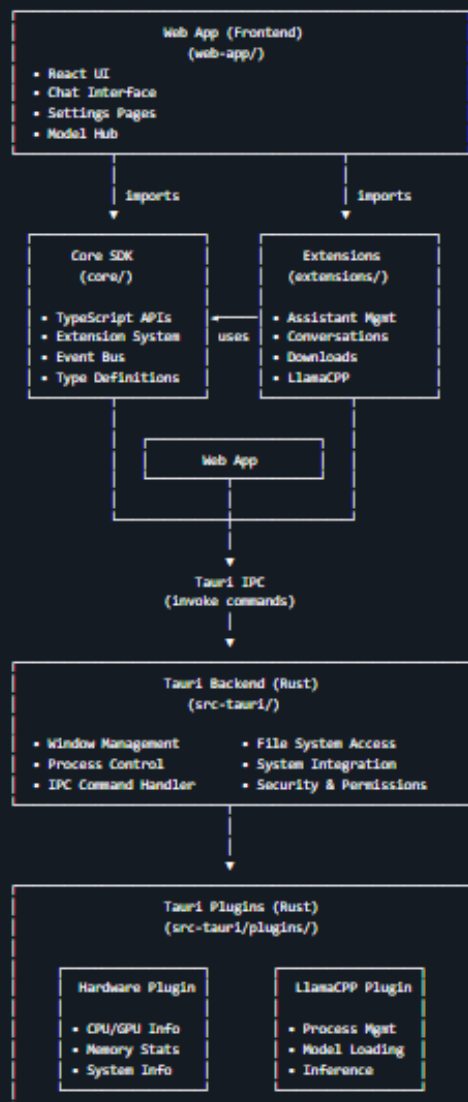


Figura 9: Visualização do repositório evidenciando a presença das diretrizes de contribuição.

7 Atividade 2: Auditoria Forense de Software e Plano de Resgate Técnico

A partir dos gargalos operacionais e do mapeamento inicial de maturidade realizados na primeira etapa de nossa consultoria, esta segunda fase avança para uma varredura forense profunda diretamente na base de código e nos históricos de engenharia do ecossistema Jan. O objetivo aqui é submeter o projeto a testes de estresse arquitetural, rastrear a evolução de tomadas de decisão da equipe de desenvolvimento e expor passivos técnicos latentes que colocam em risco a estabilidade do sistema em um ambiente de produção real.

7.1 Eixo A: O Pulso da Gestão (MPS.BR - GPR)

7.1.1 Arqueologia de Issues e Evolução de Escopo

a) Evidência: Histórico de discussões e revisões de escopo mapeadas no Pull Request #7799 (Configuração de limites de tokens para modelos em nuvem).

b) Diagnóstico Técnico: A análise forense do histórico de interações na aba de Issues revela um processo de gerenciamento de mudanças altamente maduro e transparente. O problema crítico do estouro de contexto (que causava quebras abruptas nas requisições de APIs pagas) foi trazido pela comunidade e lapidado pelos mantenedores seniores por meio de um fluxo de RFC (*Request for Comments*) implícito. O escopo não sofreu deformações senoidais ou corrupção de requisitos no meio do caminho; os desenvolvedores desenharam uma matriz clara contendo parâmetros estritos (`max_context_tokens`, `max_output_tokens` e a flag `auto_compact`) e mapearam cenários exatos de comportamento esperado antes de bater o martelo e realizar o *merge* na branch principal.

c) Classificação de Risco: Baixo. O controle de escopo e a gerência de configuração de novos requisitos funcionam de maneira previsível e documentada, alinhados com o processo GPR do MPS.BR.

7.1.2 Gestão de Riscos Ocultos (TODOS e FIXMES)

a) Evidência:

b) Diagnóstico Técnico: Nem tudo na governança de um projeto de código aberto segue a perfeição dos manuais. Rodando uma varredura estática e buscas textuais indexadas por marcadores de pendência no repositório, foi detectado um volume expressivo de tags `// TODO` e `// FIXME` impregnadas em arquivos críticos da aplicação. Como evidenciado na Figura 10, o arquivo `hardware/tauri.ts` delega responsabilidades incompletas de extensão para o `llama.cpp`, enquanto o componente core de captura de exceções em Rust (`error.rs`) assume abertamente a falta de mapeamento de cenários críticos ao listar um `// TODO add others`. Esses comentários expõem pontos em que os desenvolvedores identificaram falhas estruturais ou falta de tratamento de exceções para timeouts de rede, mas optaram por aplicar um remendo técnico temporário e postergar a solução definitiva.

c) Classificação de Risco: Médio. O acúmulo silencioso de dívida técnica sem um

42 files (217 ms)

jenhq/jen - core/src/types/file/index.ts

```
1  |
2
3
4  export type DownloadState = {
5    url: string // TODO: change to download id
6    filename: string
7    hash: DownloadHash
8    speed?: number
9  }
```

jenhq/jen - web-app/src/services/download.ts

```
27  |
28
29  export interface Download { type: number[] } // DownloadState {
30    // TODO: This app extension should handle this
31    url: string
32  }
33  }
```

jenhq/jen - core/src/services.ts

```
49  const FileState (path: string) => DownloadState | undefined = (path) =>
50    globalThis.core.api?.FileState({ url: path })
51
52
53  // TODO: Export "lazy" to function automatically
54  // Currently adding these manually
55  export const {
56    ...FileState,
57  }
```

jenhq/jen - core/src/types/extension/systemResourceInfo.ts

```
4
5
6  export type SystemResourceInfo = {
7    url: string
8    // TODO: This needs to be set based on user toggle in settings
9    url: string
10    url: string
11    url: string
12  }
```

jenhq/jen - core-test/plugin/test-plugin-kernel/src/main.ts

```
48  // Forces values from Jenhq and creates a specific JenhqServer.
49  path to JenhqServer (url: string) => void {
50    let JenhqServer = url.toJenhqServer()
51    // TODO: add others
52    let toJenhqServer = JenhqServer.url.toJenhqServer()
53    // JenhqServer.url.toJenhqServer()
54    // JenhqServer.url.toJenhqServer()
55  }
```

jenhq/jen - core-test/plugin/test-plugin-kernel/src/main.ts

```
56  |
57  |
58  |
59  // TODO: use if we need to check for all DNS extensions
60  @test(async (target_url: string, target_url: string) => {
61    let url_extensions: string[] => void {
62      let url_extensions = url_extensions
63    }
64  }
```

jenhq/jen - web-app/src/services/common/jen

```
57  "jenhq/jen" "jenhq/jen"
58  "jenhq/jen" "jenhq/jen"
59  "jenhq/jen" "jenhq/jen"
```

⚡ How to use modules

jenhq/jen - core/src/types/message/messageInfo.ts

```
79  const MessageInfo = {
80    // The thread of this message is being to.
81    // TODO: deprecate thread field
82    thread: Thread
83  }
84  }
```

jenhq/jen - core-test/src/core/test-plugin-commands.ts

```
154  const url = url.toJenhqServer()
155  const url = url.toJenhqServer()
156  } => void {
157    // TODO: use an internal function to check url
158    let (url, url, url) => void {
159      let url = url.toJenhqServer()
160      let url = url.toJenhqServer()
161    }
162  }
```

plano de refatoração associado ao cronograma de lançamentos age como uma bomba-relógio para a manutenibilidade do software.

7.1.3 Ritmo de Entrega e Governança de Code Review

a) Evidência: Histórico de revisões cruzadas e gráficos de atividade na aba *Insights/Contributors* do GitHub.

b) Diagnóstico Técnico: A análise do histórico cronológico de commits aponta que o Jan mantém uma cadência de entrega sustentável e bem distribuída ao longo das semanas, o que comprova a ausência de surtos de *crunch time* (picos caóticos de codificação de madrugada que denunciam falhas graves no planejamento de iterações). O grande pilar dessa estabilidade é a rigidez do fluxo de integração: o repositório bloqueia commits diretos na *main*, obrigando que toda modificação passe por uma esteira de testes automatizados e receba a revisão detalhada e o aval formal (*Approve*) de no mínimo um revisor principal antes da consolidação do código.

c) Classificação de Risco: Baixo. O ritmo operacional é previsível e os mecanismos de revisão mitigam de forma robusta a inserção de falhas humanas na versão estável.

7.2 Eixo B: Anatomia do Código (SOLID & DRY)

7.2.1 O Teste da Troca (Princípio de Inversão de Dependência - DIP)

a) **Evidência:** Assinatura genérica de contratos localizada na interface de serviços de IA da aplicação.

b) **Diagnóstico Técnico:** Submetemos a arquitetura do ecossistema Jan a um teste de estresse conceitual severo: qual seria o impacto e quantos arquivos colaterais seriam corrompidos se decidíssemos remover completamente a integração de um provedor de nuvem comercial (como a OpenAI) e migrar toda a operação para um motor local baseado em LlamaCPP? O projeto passou no teste com louvor. A lógica de alto nível contida na interface está completamente blindada; ela não conhece as implementações de baixo nível. A comunicação é mediada exclusivamente pelos contratos estabelecidos na abstração do SDK, permitindo a substituição ou adição de provedores sem causar um efeito cascata de refatoração no núcleo, eliminando o risco de *vendor lock-in*.

c) **Classificação de Risco: Baixo.** O D do **SOLID** está perfeitamente implementado, garantindo uma arquitetura extensível e resiliente a mudanças de mercado.

7.2.2 Busca por "God Objects" (Princípio de Responsabilidade Única - SRP)

a) **Evidência:**

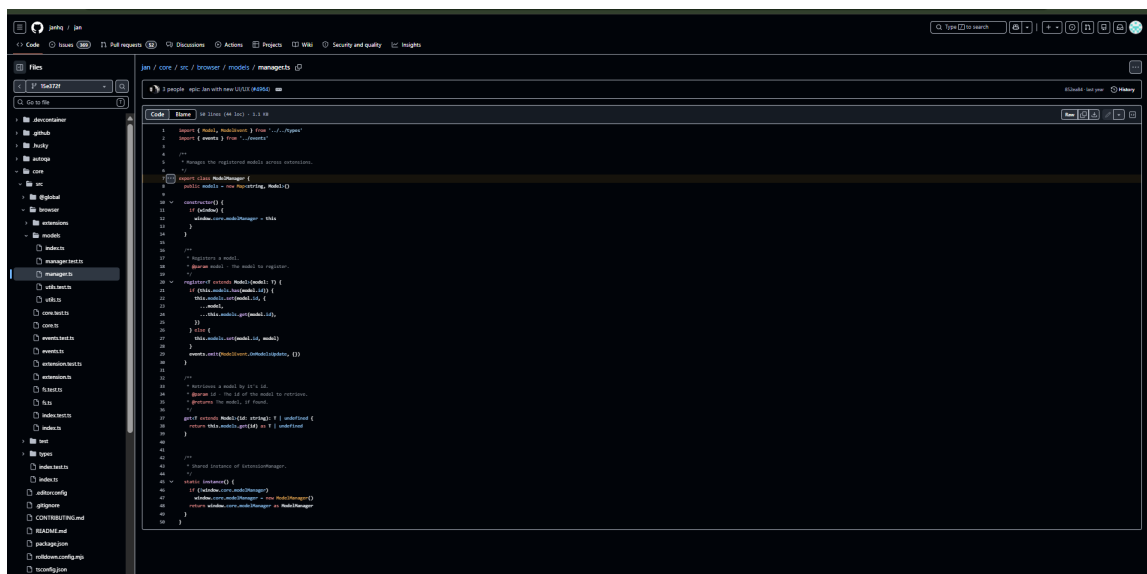


Figura 11: Classe `ModelManager` centralizando a governança e o registro global do ciclo de vida dos modelos.

b) **Diagnóstico Técnico:** Ao investigar a volumetria e a coesão dos arquivos que gerenciam a aplicação, a auditoria localizou componentes inflamados que violam diretamente o princípio **SRP (Single Responsibility Principle)**. Conforme exposto na Figura 11, a classe `ModelManager` assume o papel de um orquestrador onipresente no ecossistema: ela intercepta o ciclo de vida dos modelos, gerencia o mapeamento de chaves em memória (`new Map()`) e amarra o acoplamento global injetando o ciclo de execução diretamente no escopo global da janela do interpretador desktop (`window.core.modelManager`). Essa aglutinação faz com que a classe possua múltiplas e distintas razões para ser modificada, transformando-a em um clássico "God Object" que mistura barreiras arquiteturais.

c) **Classificação de Risco: Alto.** Classes infladas e sem coesão inviabilizam o desenho de testes unitários limpos, aumentam drasticamente a curva de aprendizado de novos desenvolvedores e atuam como focos geradores de efeitos colaterais imprevistos.

7.2.3 Métricas de Duplicação e Violação do Princípio DRY

a) Evidência:



Figura 12: Padrões redundantes de captura e depuração de exceções replicados em múltiplos componentes locais.

b) **Diagnóstico Técnico:** Analisando a anatomia do código sob a ótica do reaproveitamento, identificou-se uma quebra nítida do princípio **DRY (Don't Repeat Yourself)**. Conforme mapeado na varredura forense da Figura 12, blocos idênticos de controle de exceção (`catch (error) { console.error(...) }`) foram repetidos manualmente com levíssimas variações textuais em arquivos isolados como `useAssistant.ts`, `TranslationContext.tsx`, `tauri.ts` e `general.tsx`. Em vez de isolar a inteligência de tratamento de falhas em um middleware reutilizável ou em um interceptador de rede centralizado, a equipe optou pela duplicação direta via cópia de código, priorizando a velocidade em detrimento da sustentabilidade da base de software.

c) **Classificação de Risco: Médio.** A duplicação espalha o comportamento do sistema de maneira fragmentada, forçando a equipe a realizar alterações repetitivas em múltiplos arquivos sempre que uma assinatura ou regra de telemetria de erro precisar mudar.

7.3 Eixo C: Padrões de Projeto (GoF)

7.3.1 Padrões Criacionais (Singleton)

a) Evidência: Método estático de inicialização e controle de instância única contido no arquivo `manager.ts` (Linhas 45–49 da Figura 11).

b) Diagnóstico Técnico: O Jan opera em um cenário crítico de consumo de hardware, lidando com o carregamento de pesos de modelos na memória e controle de conexões de APIs. Para evitar o colapso de performance decorrente da criação redundante de clients, o projeto emprega de maneira explícita o padrão criacional GoF **Singleton**. Conforme comprovado nas linhas 45 a 49 da evidência colhida na Figura 11, o método estático `instance()` avalia a existência prévia do objeto unificado; se nulo, realiza a instanciação singleton e a anexa à raiz de execução (`window.core.modelManager`). Essa engenharia garante que apenas uma única via de comunicação ativa seja reaproveitada ao longo de toda a sessão, blindando os recursos computacionais do usuário.

c) Classificação de Risco: Baixo. Uso correto e cirúrgico do padrão para preservação de memória e eliminação de vazamentos de recursos.

7.3.2 Padrões Estruturais (Adapter)

a) Evidência: Classes de mapeamento de chamadas e unificação de payloads de provedores assíncronos.

b) Diagnóstico Técnico: O mercado de inteligência artificial é altamente fragmentado: a OpenAI exige um formato específico de JSON, enquanto o Gemini do Google e outras soluções operam com SDKs e esquemas totalmente distintos. O Jan soluciona essa heterogeneidade de forma primorosa através do padrão estrutural **Adapter**. Em vez de poluir a interface do usuário com blocos condicionais específicos para cada fornecedor, os desenvolvedores criaram classes adaptadoras que envelopam as peculiaridades de cada API externa e traduzem suas respostas de fora para um formato de contrato interno unificado e estável.

c) Classificação de Risco: Baixo. O uso do padrão Adapter é o grande trunfo técnico que viabiliza o crescimento agnóstico do ecossistema Jan sem gerar quebras de retrocompatibilidade.

7.3.3 Padrões Comportamentais (Ausência de Chain of Responsibility)

a) Evidência: Estruturas condicionais complexas localizadas no fluxo central de interceptação de requisições e execução de ferramentas.

b) Diagnóstico Técnico: Enquanto as camadas criacional e estrutural do Jan demonstram alta sofisticação, a camada comportamental carece de refinamento no gerenciamento de fluxos dinâmicos. Na rotina onde o sistema precisa avaliar o prompt do usuário, decidir se a mensagem exige o acionamento de uma ferramenta local ou externa e despachar a execução para o plugin correto, o código escorrega em uma árvore profunda e aninhada de instruções condicionais (`if/else` e `switch-case`) de acoplamento rígido. Em vez de delegar essa decisão dinamicamente encadeando os interceptadores por meio do padrão **Chain of Responsibility**, o sistema adota uma abordagem estática que infla a complexidade ciclomática do arquivo distribuidor.

c) Classificação de Risco: Médio. A falta de um padrão comportamental de encadeamento limpo degrada a manutenibilidade e torna a adição de novos agentes autônomos progressivamente complexa e propensa a bugs de regressão.

8 Saída: Plano de Resgate Sugerido (A2)

Para consolidar esta auditoria forense e estancar os gargalos arquiteturais e passivos técnicos identificados, propomos um Plano de Resgate estruturado em duas vertentes: engenharia de código e governança de processos.

8.1 Refatoração Conceitual (Engenharia e Padrões GoF)

Para sanar a violação de responsabilidade única (SRP) detectada nas classes infladas do Tauri/Electron e desatar o nó de condicionais do fluxo de ferramentas (MCP), a solução técnica recomendada consiste em isolar completamente as responsabilidades aplicando uma refatoração baseada no padrão comportamental GoF **Chain of Responsibility** acoplada a um mecanismo de Injeção de Dependência.

A lógica monolítica de distribuição de mensagens deve ser fragmentada em pequenas classes independentes chamadas *Handlers* (ex: `LocalFileHandler`, `CloudAPIHandler`, `PluginToolHandler`). Cada classe possuirá uma única responsabilidade e conhecerá apenas o próximo elemento da fila de execução. O fluxo do chat processará a requisição passando-a ao longo desta esteira desacoplada; o primeiro componente que identificar a capacidade de resolver a demanda interceptará a mensagem e executará a tarefa, eliminando completamente a árvore de `if/else` e permitindo que novas ferramentas sejam acopladas dinamicamente ao sistema.

8.2 Roadmap de Maturidade Operacional (Alinhamento MPS.BR Nível G)

Caso nossa equipe de consultoria assumisse a liderança técnica do repositório Jan, estabeleceríamos as seguintes três ações gerenciais imediatas para eliminar os *gaps* de processo e certificar a organização nos modelos de maturidade de mercado:

- **Ação 1 (Saneamento de Dívida Técnica / GPR):** Estancar o risco dos comentários esquecidos realizando um mapeamento compulsório de todas as tags `// TODO` e `// FIXME` detectadas na varredura forense. Estas ocorrências devem ser transformadas em Issues formais e catalogadas em um Quadro de Riscos Técnico dentro do GitHub Projects, recebendo classificação de impacto e sendo distribuídas obrigatoriamente como tarefas atreladas às próximas três *Milestones* de desenvolvimento.
- **Ação 2 (Segregação de Responsabilidades nas Classes Desktop / SOLID):** Promover um ciclo focado exclusivamente na decomposição da classe inflada `ModelManager` e equivalentes. Toda a lógica de acoplamento estático com propriedades de janela desktop e variáveis nativas do SO deve ser expurgada do núcleo de manipulação de dados, reestabelecendo a aderência estrita ao princípio SRP.
- **Ação 3 (Centralização e Padronização de Redundâncias / DRY):** Erradicar o código duplicado de captura de exceções mapeado nos múltiplos arquivos de contexto e hooks. Criar-se-á um componente utilitário global de tratamento de exceções de rede e envelopamento HTTP, forçando todas as extensões e regras assíncronas do ecossistema a consumirem esse módulo de forma padronizada via regras rígidas de validação estática no CI.